

UNIVERSIDAD DON BOSCO

Desarrollo de Software Propietario · DSP404



SEMANA 8

Introducción al Desarrollo Web

ASP.NET Core · Razor Pages · Formularios · Validación

Cliente vs Servidor

Razor Pages

Formularios

Validación

¿Qué lograrás al terminar esta clase?

01

Explicas cliente vs servidor

Qué corre en el navegador y qué corre en el servidor, y por qué esa separación existe.

02

Distingues estático de dinámico

HTML fijo vs páginas generadas por código en tiempo real, y cuándo conviene cada uno.

03

Creas tu primer proyecto Razor Pages

Estructura de carpetas, Program.cs y el modelo de páginas de ASP.NET Core.

04

Construyes un formulario con validación

Data Annotations y validación del lado del servidor, la que de verdad protege.

05

Manejas errores y excepciones

try/catch y páginas de error amigables para el usuario final.

Contenido de la Clase

Semana 8 — Introducción al Desarrollo Web con ASP.NET Core

01

La Web: Cliente y Servidor

Cómo funciona internet por dentro.

02

Estático vs Dinámico

HTML fijo vs generado por código.

03

ASP.NET Core y Razor Pages

El framework y su estructura.

04

Formularios y Validación

Capturar y validar datos del usuario.

05

Manejo de Errores

try/catch y páginas de error.



SECCIÓN 1

La Web

Cliente y Servidor — cómo funciona internet

40'

¿Qué pasa cuando abres una página web?



Toda la web funciona con un modelo PETICIÓN → RESPUESTA. Tu navegador (cliente) PIDE algo, un servidor lo PROCESA y RESPONDE. Entender esto es la base de todo el desarrollo web.

CLIENTE

Navegador (Chrome, Edge...)

HTML, CSS, JS

① PETICIÓN (Request)

GET /productos



SERVIDOR

ASP.NET Core

C#, base de datos, lógica del negocio

② RESPUESTA (Response)

HTML generado



Analogía: tú (cliente) pides un platillo al mesero. La cocina (servidor) lo prepara y te lo trae — no ves cómo lo cocinan, solo recibes el resultado en tu mesa.

¿Por qué la web funciona así?

El modelo cliente-servidor no es casualidad: se diseñó así porque resuelve varios problemas a la vez. Estas son las razones detrás del diseño.



Datos centralizados

La información vive en un solo lugar (el servidor). Todos los clientes ven datos consistentes y actualizados.



Lógica protegida

Las reglas de negocio y el acceso a la base de datos quedan en el servidor, lejos de miradas curiosas en el navegador.



Muchos clientes, un servidor

Miles de navegadores distintos pueden hablar con el mismo servidor sin que este dependa de qué dispositivo usan.



Actualizar sin reinstalar

Si cambias el servidor, todos los usuarios ven la mejora de inmediato — nadie descarga nada nuevo.



Cliente ligero

El navegador solo necesita saber pedir y mostrar; el trabajo pesado (cálculos, datos) lo hace el servidor.



Estándar universal

HTTP es el mismo protocolo en todo el mundo: cualquier cliente entiende a cualquier servidor.

¿Qué corre en el cliente y qué en el servidor?

■ En el CLIENTE (navegador)

- ✓ HTML — estructura de la página
- ✓ CSS — estilos y diseño
- ✓ JavaScript — interactividad
- ✓ Lo que el usuario VE y toca
- ✓ Ya lo conocen de clases previas

⚙ En el SERVIDOR (ASP.NET)

- ✓ C# — lógica del programa
- ✓ Acceso a base de datos
- ✓ Validación de seguridad
- ✓ Generación del HTML dinámico
- ✓ Lo que aprenden a partir de hoy



Regla de oro de seguridad: NUNCA confíes solo en la validación del cliente (JavaScript). El usuario puede saltársela. La validación del servidor (C#) es la que protege de verdad — lo veremos a fondo en el bloque de Formularios.

HTTP — el idioma entre cliente y servidor

 HTTP (HyperText Transfer Protocol) es el conjunto de reglas con que el cliente y el servidor se comunican. Cada petición usa un VERBO que dice qué quieres hacer.

GET**Pedir / leer datos**

Ver una página, cargar una lista de productos

POST**Enviar / crear datos**

Enviar un formulario, registrar un usuario

PUT**Actualizar datos**

Editar un producto existente

DELETE**Borrar datos**

Eliminar un registro

 Por ahora nos enfocamos en GET (mostrar páginas) y POST (enviar formularios). PUT y DELETE los verán al construir APIs en la semana 15.

Anatomía de una petición HTTP real



Cada petición y respuesta trae más que solo datos: incluyen encabezados (headers) y un código de estado que dice cómo salió todo.

Petición (Request)

```
GET /productos HTTP/1.1
Host: mitienda.com
Accept: text/html
User-Agent: Chrome/120.0
```

(sin cuerpo – GET no envía datos)

Respuesta (Response)

```
HTTP/1.1 200 OK
Content-Type: text/html

<html>
  <body>Lista de productos...</body>
</html>
```

Códigos de estado más comunes

200

OK — todo salió bien

302

Redirección a otra página

404

No encontrado

500

Error interno del servidor



ASP.NET Core genera estos encabezados y códigos de estado automáticamente. Cuando algo falla, el código de estado es lo primero que debes revisar.



SECCIÓN 2

Estático vs Dinámico

HTML fijo vs páginas generadas por código

20'

Web estática vs Web dinámica

Web ESTÁTICA

El HTML es fijo. Todos los usuarios ven exactamente lo mismo — el archivo .html ya existe en el servidor tal cual.

- Páginas .html escritas a mano
- El contenido NO cambia
- Mismo resultado para todos
- Ej: una página 'Acerca de'

index.html

```
<h1>Bienvenido</h1>
<p>Somos una tienda.</p>
```

Web DINÁMICA

El HTML se GENERA en el momento, con código C#. Cada usuario puede ver contenido distinto según sus datos.

- Páginas generadas por código
- El contenido cambia según datos
- Personalizado por usuario
- Ej: 'Hola, Ana' con TUS pedidos

Razor: .cshtml

```
<h1>Hola, @Model.Nombre</h1>
<p>Tienes @Model.Pedidos pedidos.</p>
```



La @ en Razor (@Model.Nombre) es la clave de lo dinámico: el servidor reemplaza esa expresión con el dato real antes de mandar el HTML al navegador.

¿Cuándo conviene cada una?



Ninguna es 'mejor' de forma absoluta: cada una resuelve un problema distinto. Elegir bien depende de qué necesita tu sitio.



Estática: velocidad y simplicidad

No hay cálculos en el servidor por cada visita, así que carga muy rápido y es barata de alojar.



Estática: contenido que no cambia

Ideal para landing pages, portafolios, documentación — información igual para cualquier visitante.



Estática: menor superficie de ataque

Al no ejecutar código en el servidor por petición, hay menos puntos donde algo pueda salir mal.



Dinámica: personalización

Necesaria cuando cada usuario debe ver algo distinto: su carrito, su perfil, sus pedidos.



Dinámica: datos en vivo

Cuando el contenido depende de una base de datos que cambia constantemente (inventario, precios, noticias).



Dinámica: lógica de negocio

Cuando hay reglas que procesar: calcular un total, validar un login, generar un reporte.



SECCIÓN 3

ASP.NET Core

El framework y Razor Pages

45'

¿Qué es ASP.NET Core? — el concepto



ASP.NET Core es el framework de Microsoft para construir aplicaciones web con C#. Es gratuito, multiplataforma y muy rápido. Razor Pages es la forma más simple de empezar.



Gratuito y open source

Sin licencias. Corre en Windows, Linux y macOS.



Muy rápido

De los frameworks web más veloces del mercado.



Razor Pages

Modelo simple: cada página tiene su HTML y su código C#.



MVC (más adelante)

Para apps grandes. Lo verán en la semana 13.



Web APIs

Para servicios y apps móviles. Semana 15.



Integra bases de datos

Con Entity Framework Core. Semana 14.



El camino del curso: hoy Razor Pages (simple) → luego MVC → EF Core → Web APIs. Todo se construye sobre la misma base: ASP.NET Core.

Un poco de historia — ¿por qué así?



ASP.NET Core no nació de la nada: es resultado de más de 20 años de Microsoft simplificando cómo se construyen apps web. Conocer el porqué ayuda a entender sus decisiones de diseño.

1

Web Forms (2002)

Intentaba ocultar HTML/HTTP con controles tipo escritorio. Potente, pero pesado y difícil de probar.

2

.NET Framework + MVC (2009)

Separó vista y lógica (patrón MVC), pero solo corría en Windows y era más complejo de aprender.

3

ASP.NET Core (2016)

Reescrito desde cero: multiplataforma, ligero y rápido. Nace pensando en la nube y en Linux.

4

Razor Pages (2017)

Una capa más simple sobre ASP.NET Core: cada página es autónoma, ideal para aprender y para apps pequeñas-medianas.



Por eso empiezan con Razor Pages: es el punto de entrada más simple al mismo motor que usa MVC — lo que aprendan hoy no se pierde, se expande.

Estructura de un proyecto Razor Pages

MiPrimeraWebApp/

```
Pages/  
  Index.cshtml  
  Index.cshtml.cs  
  Privacy.cshtml  
  Shared/  
    _Layout.cshtml  
wwwroot/  
  css/   js/   img/  
Program.cs  
appsettings.json
```

Pages/

Aquí van todas tus páginas web

.cshtml

El HTML + Razor (lo que el usuario ve)

.cshtml.cs

El código C# de esa página (PageModel)

wwwroot/

Archivos estáticos: CSS, JS, imágenes

Program.cs

Configura y arranca la aplicación

_Layout.cshtml

Plantilla común para todas las páginas



Clave: cada página son DOS archivos. Index.cshtml (vista) e Index.cshtml.cs (lógica). Esto se llama 'code-behind' — la vista muestra, el .cs decide.

El par de archivos: Vista + PageModel

i Cada página Razor son DOS archivos que trabajan juntos: el .cshtml (la vista — qué se ve) y el .cshtml.cs (el PageModel — la lógica en C#).

Index.cshtml (la Vista)

```
@page
@model IndexModel

<h1>@Model.Saludo</h1>

<p>La hora actual es:
    @Model.HoraActual</p>

@foreach (var p in Model.Productos)
{
    <div>@p</div>
}
```

Index.cshtml.cs (el PageModel)

```
public class IndexModel : PageModel
{
    public string Saludo { get; set; }
    public string HoraActual { get; set; }
    public List<string> Productos { get; set; }

    public void OnGet() // se ejecuta al cargar
    {
        Saludo = "Bienvenido a la tienda";
        HoraActual = DateTime.Now.ToString("HH:mm");
        Productos = new() { "Laptop", "Mouse" };
    }
}
```

⌘ El @model conecta los dos archivos. @Model.Saludo en la vista muestra el valor que el PageModel asignó en OnGet(). El servidor une ambos y manda el HTML final.

¿Por qué separar vista y lógica?



Podrían escribir C# directamente dentro del HTML en un solo archivo. ASP.NET Core obliga a separarlo por buenas razones — este principio se llama 'separación de responsabilidades'.



Código más ordenado

El diseño (HTML/CSS) y la lógica (C#) no se mezclan; cada archivo tiene un solo propósito claro.



Más fácil de probar

El PageModel se puede probar con unit tests sin necesidad de renderizar HTML ni abrir un navegador.



Trabajo en equipo

Un diseñador puede tocar el .cshtml mientras un backend developer trabaja el .cshtml.cs, sin pisarse.



Reutilización

La misma lógica de PageModel puede alimentar distintas vistas si el diseño cambia.



Errores más fáciles de rastrear

Si algo falla en los datos, buscas en el .cs. Si falla en el diseño, buscas en el .cshtml.



Un principio, no una regla de Razor

Este mismo principio (separar qué se ve de qué se decide) aparece en casi todo framework web moderno.

Program.cs — el corazón de la aplicación

Program.cs — punto de entrada (.NET 10)

```
var builder = WebApplication.CreateBuilder(args);

// 1. Registrar servicios
builder.Services.AddRazorPages();

var app = builder.Build();

// 2. Configurar el pipeline
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
}

app.UseStaticFiles(); // CSS, JS, imágenes
app.UseRouting();     // habilitar rutas
app.MapRazorPages();  // conectar páginas

// 3. Arrancar
app.Run();
```

1

CreateBuilder

Prepara la configuración de la app.

2

AddRazorPages

Activa el sistema de páginas Razor.

3

UseStaticFiles

Permite servir CSS/JS/imágenes.

4

MapRazorPages

Conecta las URLs con las páginas.

5

app.Run()

Arranca el servidor web.



No necesitan memorizar Program.cs ahora. Solo entender el flujo: configurar servicios → configurar pipeline → arrancar.



SECCIÓN 4

Formularios

Capturar y validar datos del usuario

35'

¿Cómo viajan los datos de un formulario?

i Cuando el usuario llena un formulario y presiona 'Enviar', el navegador hace una petición POST con los datos. El método `OnPost()` en el `PageModel` los recibe.



🚩 `OnGet()` se ejecuta cuando se CARGA la página (mostrar el formulario vacío). `OnPost()` se ejecuta cuando se ENVÍA (procesar los datos). Son dos momentos distintos del mismo par de archivos.

🔄 Piénsalo como abrir una puerta (`OnGet`, solo mirar) vs entregar un paquete por esa puerta (`OnPost`, algo se procesa). La URL es la misma; la acción es distinta.

Formulario con Razor — vista y modelo

Registro.cshtml (formulario)

```
@page
@model RegistroModel

<form method="post">
  <label>Nombre:</label>
  <input asp-for="Nombre" />
  <span asp-validation-for="Nombre"></span>

  <label>Email:</label>
  <input asp-for="Email" />

  <button type="submit">Registrar</button>
</form>

@if (Model.Exito)
{
  <p>[OK] ¡Registro exitoso!</p>
}
```

Registro.cshtml.cs (PageModel)

```
public class RegistroModel : PageModel
{
  [BindProperty]
  public string Nombre { get; set; }

  [BindProperty]
  public string Email { get; set; }

  public bool Exito { get; set; }

  public void OnGet() { } // muestra el form

  public IActionResult OnPost()
  {
    if (!ModelState.IsValid)
      return Page(); // hay errores
    Exito = true;
    return Page();
  }
}
```



[BindProperty] conecta el campo del formulario con la propiedad C#. asp-for genera el HTML correcto automáticamente. ModelState.IsValid revisa si pasó la validación.

Data Annotations — validar sin escribir if

i Las Data Annotations son etiquetas [entre corchetes] que pones sobre una propiedad para definir sus reglas de validación. ASP.NET las revisa automáticamente.

RegistroModel.cs

```
[BindProperty]
[Required(ErrorMessage = "Obligatorio")]
[StringLength(50, MinimumLength = 3)]
public string Nombre { get; set; }

[BindProperty]
[Required]
[EmailAddress(ErrorMessage = "Inválido")]
public string Email { get; set; }

[BindProperty]
[Range(18, 120)]
public int Edad { get; set; }
```

[Required]

Campo obligatorio

[StringLength]

Largo máx/mín del texto

[EmailAddress]

Formato de email válido

[Range(a,b)]

Número en un rango

[Phone]

Formato de teléfono

[Compare]

Comparar 2 campos (contraseñas)

i Ventaja enorme: en lugar de escribir 20 líneas de if para validar, pones una etiqueta. ASP.NET genera los mensajes de error y los muestra automáticamente.

Defensa en profundidad — dos capas



Las Data Annotations validan en el servidor, pero ASP.NET también genera validación automática en JavaScript para el cliente. ¿Por qué las dos, si con una ya funciona?

Validación en el CLIENTE

- Responde al instante, sin esperar al servidor
- Mejora la experiencia de usuario (UX)
- Se puede desactivar o saltar fácilmente
- NUNCA es suficiente por sí sola

Validación en el SERVIDOR

- El usuario no puede evitarla ni manipularla
- Es la última línea de defensa real
- Protege la base de datos y la lógica
- Un poco más lenta (viaja al servidor)



Se usan las dos juntas: el cliente da respuesta rápida y buena experiencia; el servidor garantiza seguridad real. Esto se llama 'defensa en profundidad' — nunca confíes en una sola capa.



SECCIÓN 5

Manejo de Errores

Excepciones y páginas de error

20'

¿Qué es una excepción? — try / catch

i Una excepción es un error que ocurre mientras el programa corre (dividir por cero, archivo no encontrado, conexión perdida). Si no se maneja, el programa se cae. try/catch lo atrapa.

Anatomía del try/catch

```
try
{
    // Código que PODRÍA fallar
    int resultado = 100 / divisor;
    var datos = LeerArchivo("config.txt");
    GuardarEnBaseDeDatos(datos);
}
catch (DivideByZeroException ex)
{
    // Maneja un error específico
    Console.WriteLine("No se puede dividir");
}
catch (Exception ex)
{
    // Maneja cualquier otro error
    Console.WriteLine($"Error: {ex.Message}");
}
finally
{
    // SIEMPRE se ejecuta
    CerrarConexion();
}
```

1**try**

El código que podría fallar.

2**catch**

Qué hacer si hay error.

3**finally**

Siempre se ejecuta al final.

4**ex.Message**

El mensaje del error.



Atrapa primero las excepciones específicas (DivideByZeroException) y al final la general (Exception). El orden importa.

Páginas de error amigables en la web



En una web real, NUNCA muestres el error técnico al usuario (revela información y se ve mal). Muestra una página de error amigable y registra el detalle en un log para ti.

✗ Lo que NO debes mostrar

```
System.NullReferenceException:  
Object reference not set  
at MyApp.Productos.OnGet()  
in C:\Users\dev\Productos.cs:42  
[Stack trace completo visible]
```

✓ Lo que SÍ debes mostrar

⚠ ¡Ups! Algo salió mal

Estamos trabajando en ello.
Intenta de nuevo en unos minutos.

```
if (!app.Environment.IsDevelopment())  
    app.UseExceptionHandler("/Error"); // ← muestra Error.cshtml en producción
```



El error técnico (stack trace) es para los desarrolladores en los logs, NO para el usuario final.

Buenas prácticas de manejo de errores



Manejar errores no es solo evitar que la app se caiga: es una responsabilidad profesional. Estas prácticas separan un proyecto de estudiante de uno listo para producción.



Registra todo (logging)

Guarda el detalle del error en un log — lo necesitarás para diagnosticar sin exponerlo al usuario.



Nunca expongas detalles técnicos

Stack traces y mensajes internos son información valiosa para un atacante.



Atrapa lo específico primero

Un catch genérico oculta la causa real; los específicos permiten reaccionar distinto según el error.



Usa finally para limpiar

Cerrar conexiones o archivos debe pasar SIEMPRE, haya error o no.



No uses try/catch para controlar flujo normal

Las excepciones son para lo inesperado, no para decisiones cotidianas del programa.



Piensa en el usuario final

Un mensaje claro y humano genera confianza; un error críptico la destruye.



UNIVERSIDAD DON BOSCO

¡GRACIAS!

DSP404 · Introducción al Desarrollo Web